

Capitolo 5a - CPU Scheduling

Status Done

L'obiettivo della multiprogrammazione è massimizzare l'utilizzo della CPU (o dei core, nei sistemi multicore).

- **Problema:** in un sistema semplice, quando un processo è in attesa del completamento di un'operazione I/O, la CPU rimarrebbe inattiva, sprecando tempo.
- **Soluzione:** si tengono più processi contemporaneamente in memoria. Quando un processo deve attendere un evento (es. I/O), il SO gli sottrae il controllo della CPU per assegnarlo a un altro processo pronto per l'esecuzione.
- **Funzione:** lo scheduling è la funzione primaria del SO che decide come assegnare tutte le risorse (memoria, I/O, ecc.). Lo scheduling della CPU è l'aspetto più centrale e critico di questa gestione, mirato a mantenere il processore costantemente occupato.

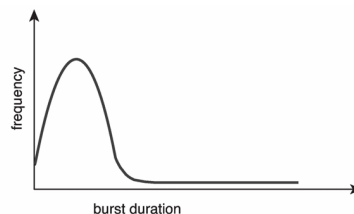
L'efficacia dello scheduling della CPU (cioè la sua capacità di mantenere la CPU utilizzata) è possibile perché i processi hanno la proprietà di **alternare continuamente** due stati:

1. **Fase di elaborazione (CPU Burst):** una sequenza di operazioni eseguite direttamente dalla CPU.
 2. **Fase di attesa (I/O Burst):** una sequenza di operazioni in cui il processo attende il completamento di richieste di I/O.
- L'esecuzione di un processo è una serie alternata di CPU burst e I/O burst, fino all'ultimo CPU burst che termina con la richiesta di terminazione del processo.

Le durate dei CPU burst non sono uniformi, ma seguono generalmente una curva di frequenza esponenziale o iperesponenziale:

- Molte sequenze brevi: la maggior parte dei CPU burst è di breve durata.
- Poche sequenze lunghe: ci sono poche sequenze di operazioni molto lunghe.
- Processi I/O-Bound: producono prevalentemente molte sequenze di CPU burst di breve durata (passano molto tempo in attesa di I/O).
- Processi CPU-Bound: possono produrre varie sequenze di CPU burst molto lunghe (passano molto tempo nell'elaborazione).

Questa distribuzione è cruciale perché aiuta a scegliere l'algoritmo di scheduling della CPU più appropriato.



CPU Scheduler

Lo **scheduler a breve termine** (o **scheduler della CPU**) è il componente del SO che entra in azione ogni volta che la CPU diventa inattiva.

- **Funzione:** sceglie quale tra i processi pronti per l'esecuzione, presenti nella ready queue, deve essere assegnato alla CPU.
- **Ready Queue:** non è necessariamente una semplice coda FIFO, può essere implementata come una coda con priorità, un albero o una lista. Gli elementi gestiti sono generalmente i Process Control Block (PCB) dei processi.

Le decisioni di scheduling sono prese in quattro circostanze, ma le scelte cruciali si hanno nei casi 2 e 3:

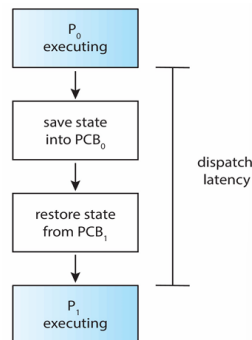
Circostanza (Decisione di Scheduling)	Descrizione
1. Esecuzione → Attesa	Processo in esecuzione chiede I/O o <code>wait()</code> e si blocca.
2. Esecuzione → Pronto	Interruzione (es. scadenza del tempo) sposta il processo da esecuzione a pronto.
3. Attesa → Pronto	L'I/O richiesto termina e il processo torna pronto per l'esecuzione.
4. Processo Termina	Il processo finisce l'esecuzione.

- **Scheduling senza prelazione (Nonpreemptive / Cooperativo):** lo scheduling avviene solo nei casi 1 e 4. Una volta assegnata la CPU a un processo, questo la mantiene finché non la rilascia volontariamente (termina o va in attesa).
- **Scheduling con Prelazione (Preemptive):** lo scheduling avviene anche nei casi 2 e 3. Il SO può sottrarre la CPU a un processo in esecuzione (es. allo scadere di un tot di tempo o per l'arrivo di un processo a priorità più alta).
 - **Importanza:** quasi tutti i moderni SO (Windows, macOS, Linux) usano lo scheduling con prelazione.
 - **Rischio:** la prelazione può causare problemi di **race condition** (incoerenza dei dati) quando i processi condividono i dati, un problema risolto con meccanismi come i **lock mutex**.

Dispatcher

Il dispatcher è il modulo che esegue l'azione fisica di assegnare la CPU al processo scelto dallo scheduler.

- **Funzioni principali:**
 - Esegue il **cambio di contesto** (salvare lo stato del vecchio processo e caricare lo stato del nuovo).
 - Passa alla **modalità utente**.
 - Salta alla corretta posizione nel programma utente per riavverne l'esecuzione.
- **Latenza di dispatch:** è il tempo impiegato dal dispatcher per fermare un processo e iniziare un altro. Deve essere il più rapido possibile (per minimizzare l'overhead).
- **Cambi di contesto volontari vs. involontari:**
 - **Volontario:** il processo cede la cpu spontaneamente (es. vai in attesa di I/O).
 - **Involontario:** la cpu viene sottratta al processo (es. scadenza del tempo o prelazione per priorità).



Criteri di scheduling della CPU

I diversi algoritmi di scheduling sono confrontati e scelti in base a criteri che misurano l'efficienza del sistema e la soddisfazione dell'utente. L'obiettivo generale è massimizzare l'utilizzo e la produttività e minimizzare i vari tempi.

Questi criteri misurano quanto efficacemente la cpu sta lavorando:

- **Utilizzo della CPU (CPU Utilization):**
 - Misura la percentuale di tempo in cui la CPU è effettivamente impegnata in lavoro utile.
 - Un sistema reale dovrebbe avere un utilizzo compreso tra il 40% (basso carico) e il 90% (alto carico).
 - **Obiettivo:** massimizzare questo valore.
- **Produttività (Throughput):**
 - Misura la quantità di lavoro completato, definita come il numero di processi completati nell'unità di tempo (es. processi all'ora o al secondo).
 - **Obiettivo:** massimizzare questo valore.

Criteri di misurazione del tempo (Minimizzazione)

Questi criteri misurano il tempo percepito dagli utenti o la velocità con cui i processi vengono gestiti:

Criterio	Definizione	Obiettivo	Note Importanti
Tempo di Completamento (Turnaround Time)	L'intervallo totale tra la sottomissione di un processo e il suo completamento. È la somma dei tempi di attesa, esecuzione, I/O, ecc.	Minimizzare	Misura l'efficacia complessiva.

Tempo di Attesa (Waiting Time)	La somma degli intervalli di tempo che il processo trascorre in attesa nella ready queue .	Minimizzare	È il criterio più influenzato direttamente dall'algoritmo di scheduling.
Tempo di Risposta (Response Time)	L'intervallo tra la richiesta di un utente (in un sistema interattivo) e la produzione della prima risposta (cioè, il tempo necessario per <i>iniziare</i> la risposta).	Minimizzare	Criterio cruciale per i sistemi interattivi; non misura il tempo totale di output.

Ottimizzazione:

- Nella maggior parte dei casi, gli algoritmi cercano di ottimizzare i valori medi dei tempi (attesa, completamento, risposta).
- Per sistemi interattivi (desktop/laptop), alcuni analisti suggeriscono che sia più importante minimizzare la varianza del tempo di risposta (renderlo prevedibile e stabile) piuttosto che solo la media. Un sistema prevedibile è spesso percepito come migliore dall'utente.

Algoritmi di scheduling della CPU

Gli algoritmi di scheduling hanno il compito di selezionare quale processo, tra quelli presenti nella ready queue, deve essere eseguito dalla cpu.

Scheduling in ordine d'arrivo (FCFS - First-Come, First-Served)

È l'algoritmo più semplice, basato sul principio FIFO (First In First Out): la cpu viene assegnata al processo che l'ha richiesta per primo.

- **Implementazione:** si utilizza una coda FIFO.
- **Vantaggio:** è semplice da implementare e comprendere.
- **Svantaggio:** il tempo medio d'attesa (**waiting time**) è spesso lungo e varia notevolmente a seconda dell'ordine di arrivo dei processi.

Esempio: Si considerino tre processi (P_1, P_2, P_3) che arrivano contemporaneamente (al tempo 0) con le seguenti durate di CPU burst:

Processo	Durata (ms)
P_1	24
P_2	3
P_3	3

Scenario 1: ordine di arrivo P_1, P_2, P_3

- Tempo di attesa: P_1 attende 0 ms, P_2 attende 24 ms, P_3 attende 27 ms.
- Tempo di attesa medio: $(0 + 24 + 27)/3 = 17\text{ms}$

Scenario 2: ordine di arrivo P_2, P_3, P_1

- Tempo di attesa: P_2 attende 0 ms, P_3 attende 3 ms; P_1 attende 6 ms.
- Tempo di attesa medio: $(0 + 3 + 6)/3 = 3\text{ms}$.

⇒ Se un processo lungo arriva prima di processi brevi, crea un effetto convoglio che aumenta inutilmente il tempo di attesa medio per tutti.

Scheduling Shortest-Job-First (SJF)

Assegna la cpu al processo che ha la **più breve lunghezza della successiva sequenza di operazioni della cpu** (*shortest next CPU burst*).

- **Principio:** l'algoritmo SJF è ottimale perché rende minimo il tempo d'attesa medio per un dato insieme di processi.
- **Realizzabilità:** non è realizzabile a livello pratico, poiché è impossibile conoscere in anticipo la durata esatta del prossimo CPU burst di un processo.
- **Approssimazione:** per superare questo limite, si **predice** la durata del prossimo burst utilizzando la media esponenziale ($\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$), dove τ_n è la previsione precedente e t_n è la durata misurata del burst precedente.
Se $\alpha = 0$, la previsione non tiene conto della storia recente: $\tau_{n+1} = \tau$.
Se $\alpha = 1$, la previsione si basa solo sull'ultimo burst effettivo: $\tau_{n+1} = \tau_n$.
Comunemente, si imposta $\alpha = 1/2$.

- **Prelazione:** può essere:
 - **Senza prelazione:** il processo corrente completa il suo burst anche se ne arriva uno più breve.
 - **Con prelazione (Shortest-Remaining-Time-First, SRTF):** se un nuovo processo arriva con un tempo rimanente più breve di quello che resta al processo in esecuzione, viene effettuata la prelazione.

Esempio SJF senza prelazione:

Si considerino quattro processi che arrivano contemporaneamente con le seguenti durate di cpu burst:

Processo	Durata (ms)
P_1	6
P_2	8
P_3	7
P_4	3

Lo scheduler li ordina e li esegue nell'ordine:

1. P_4 esegue da 0 a 3 ms.
2. P_1 esegue da 3 a 9 ms.
3. P_3 esegue da 9 a 16 ms.
4. P_2 esegue da 16 a 24 ms.

Tempo di attesa medio: $(0 + 3 + 9 + 16)/4 = 7$ ms.

⇒ Eseguendo prima i processi più brevi si riduce il tempo di attesa complessivo.

Esempio SJF con prelazione:

I processi arrivano in momenti diversi e lo scheduler verifica se un nuovo processo ha un tempo rimanente più breve del processo in esecuzione.

Processo	Istante d'Arrivo	Durata (ms)
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

1. $t=0$: arriva P_1 e inizia (8 ms)
2. $t=1$: arriva P_2 (4 ms). P_1 ha 7 ms rimanenti. Poiché $4 < 7$, si effettua la prelazione: P_2 inizia l'esecuzione.
3. $t=5$: P_2 termina (durata 4 ms). Il processo più breve rimanente è P_4 (rimanenti 5 ms). P_4 inizia.
4. $t=10$: P_4 termina. Il processo più breve rimanente è P_1 (rimanenti 7 ms). P_1 inizia.
5. $t=17$: P_1 termina. Inizia P_3 (rimanenti 9 ms) e termina al tempo 26.

Tempo di attesa medio: 6,5 ms (ottenuto come $[(10-1)+(1-1)+(17-2)+(5-3)]/4$).

Scheduling circolare (Round-Robin, RR)

È simile a FCFS, ma aggiunge la prelazione per garantire che ogni processo riceva una parte equa della cpu, ideale per i sistemi *time-sharing*.

- **Meccanismo:** ogni processo riceve un piccolo intervallo fisso di tempo, chiamato **quanto di tempo** (time slice), tipicamente 10-100 millisecondi. La **ready queue è trattata come una coda circolare**.
- **Funzionamento:**
 1. Il processo ottiene la cpu e un timer viene impostato per il quanto di tempo q .
 2. Se il processo termina prima di q , rilascia la cpu.
 3. Se il tempo q scade (timer interrupt), la cpu viene sottratta al processo, che viene messo in fondo alla ready queue.
- **Prelazione:** è un algoritmo con prelazione.
- **Impatto del quanto di tempo (q):**
 - q molto lungo: RR degenera in FCFS.

- **q molto breve**: aumenta il numero di **cambi di contesto** (overhead), riducendo l'efficienza.
- **Regola empirica**: l'80% dei cpu burst dovrebbe essere più breve del quanto di tempo.

Esempio

Si considerino tre processi che arrivano al tempo 0 con le seguenti durate di CPU burst:

Processo	Durata (ms)
P_1	24
P_2	3
P_3	3

Viene utilizzato un quanto di tempo di 4 millisecondi.

I processi vengono eseguiti a turno per un massimo 4 ms ciascuno.

1. P_1 esegue 4 ms (da 0 a 4). Restano 20 ms. Viene messo in fondo alla coda.
2. P_2 esegue 3 ms (da 4 a 7). Termina prima della scadenza del quanto.
3. P_3 esegue 3 ms (da 7 a 10). Termina prima della scadenza del quanto.
4. P_1 è di nuovo in testa. Esegue 4 ms (da 10 a 14). Restano 16 ms.
5. P_1 continua a eseguire in cicli da 4 ms (4 volte, fino al tempo 30) finché non termina.

Tempo di attesa:

- P_1 : ha il suo primo blocco (0-4), ma deve attendere che P_2 e P_3 completino i loro blocchi. Quindi attende 6 ms (10-4).
- P_2 : attende 4 ms (inizia al tempo 4).
- P_3 : attende 7 ms (inizia al tempo 7).
- **Tempo di attesa medio**: $(6 + 4 + 7)/3 \approx 5.66$ ms.

Scheduling con priorità

Associa una priorità a ogni processo e assegna la cpu al processo con la **priorità più alta**. Processi con priorità uguali usano lo schema FCFS.

- **SJF come caso speciale**: l'algoritmo SJF è una forma di scheduling con priorità, dove la priorità è inversamente proporzionale alla lunghezza del prossimo cpu burst.
- **Definizione di priorità**:
 - **Interna**: basata su proprietà misurabili (limiti di tempo, memoria, ecc.).
 - **Esterna**: basata su criteri esterni (importante del processo, fondi pagati, ecc.).
- **Prelazione**: può essere con o senza prelazione.
- **Problema critico (attesa indefinita o Starvation)**: un processo a bassa priorità potrebbe non essere mai eseguito se c'è un flusso costante di processi a priorità più alta.
- **Soluzione: Invecchiamento (Aging)**, aumentare gradualmente la priorità dei processi che sono in attesa da troppo tempo.

Esempio:

Si considerino cinque processi che arrivano al tempo 0, con le seguenti durate di CPU burst e livelli di priorità (numeri di priorità bassi indicano priorità alte):

Processo	Durata (ms)	Priorità
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Lo scheduler ordina i processi in base alla priorità:

1. P_2 (priorità 1)
2. P_5 (priorità 2)

3. P_1 (priorità 3)
 4. P_3 (priorità 4)
 5. P_4 (priorità 5)
- P_2 esegue da 0 a 1 ms.
 - P_5 esegue da 1 a 6 ms.
 - P_1 esegue da 6 a 16 ms.
 - P_3 esegue da 16 a 18 ms.
 - P_4 esegue da 18 a 19 ms.

Tempo di attesa:

- P_2 : attende 0 ms.
- P_5 : attende 1 ms.
- P_1 : attende 6 ms.
- P_3 : attende 16 ms.
- P_4 : attende 18 ms.
- **Tempo di attesa medio:** $(0 + 1 + 6 + 16 + 18)/5 = 8.2$ ms.

Scheduling a code multilivello (Multilevel Queue Scheduling)

Suddivide i processi in diverse code separate, spesso in base al tipo di processo (es. foreground/interattivo vs. background/batch).

- **Struttura:** ogni coda ha la sua priorità e il suo algoritmo di scheduling (es. RR per il foreground, FCFS per il background).
- **Scheduling tra code:** si usa uno **scheduling a priorità fissa con prelazione** tra le code. Ad esempio, la coda foreground ha priorità assoluta sulla coda background.
- **Alternativa:** assegnare porzioni di tempo alla coda (es. 80% del tempo alla coda foreground, 20% alla coda background).

Esempio:

I processi vengono assegnati in modo permanente a una delle quattro code principali, stabilendo una priorità assoluta tra di esse:

Coda	Priorità	Algoritmo (Tipico)
Real-Time Processes	Più alta	Scheduling specializzato
System Processes	Alta	Scheduling specializzato
Interactive Processes	Media	Round-Robin (RR)
Batch Processes	Più bassa	FCFS

Scheduling a code multilivello con retroazione (Multilevel Feedback Queue, MLFQ)

È l'algoritmo di scheduling più generale e flessibile, che permette ai processi di spostarsi tra le code in base al loro comportamento.

- **Idea:** separare i processi in base alle loro caratteristiche di cpu burst. I processi che usano troppo tempo di cpu (cpu-bound) vengono spostati in code a priorità più bassa.
- Esempio (3 code):
 - Coda 0 (Priorità Alta): RR con $q = 8$ ms.
 - Coda 1 (Priorità Media): RR con $q = 16$ ms.
 - Coda 2 (Priorità Bassa): FCFS.
 - Un processo inizia in coda 0. Se non finisce, retrocede in coda 1, e poi in coda 2.
- **Vantaggio:** mantiene i processi I/O-bound (brevi) nelle code ad alta priorità per un'ottima reattività.
- **Prevenzione della starvation:** un processo che attende troppo a lungo in una coda a bassa priorità viene fatto salire gradualmente (*aging*).
- **Complessità:** è l'algoritmo più complesso da definire, in quanto richiede la scelta di diversi parametri (numero di code, algoritmi per ogni coda, regole di promozione/retrocessione).

Esempio:

Abbiamo uno scheduler con tre code, ognuna con una differente politica di scheduling e quanto di tempo:

Coda	Algoritmo interno	Quanto di tempo	Regola di Movimento (Retroazione)
Q_0	Round-Robin (RR)	8 ms	Se non termina in 8 ms, il processo viene retrocesso in Q_1 .
Q_1	Round-Robin (RR)	16 ms	Se non termina in 16 ms, il processo viene retrocesso in Q_2 .
Q_2	FCFS	Nessuno	I processi lunghi finiscono qui. I processi in attesa troppo a lungo possono essere promossi (invecchiamento).

⇒ I processi corti/interattivi vengono serviti rapidamente in Q_0 e Q_1 . I processi lunghi (CPU-bound) vengono "filtrati" e spostati in Q_2 , che ha la priorità di esecuzione più bassa e usa FCFS, impedendo loro di monopolizzare la cpu.